

Blockchain technology and applications

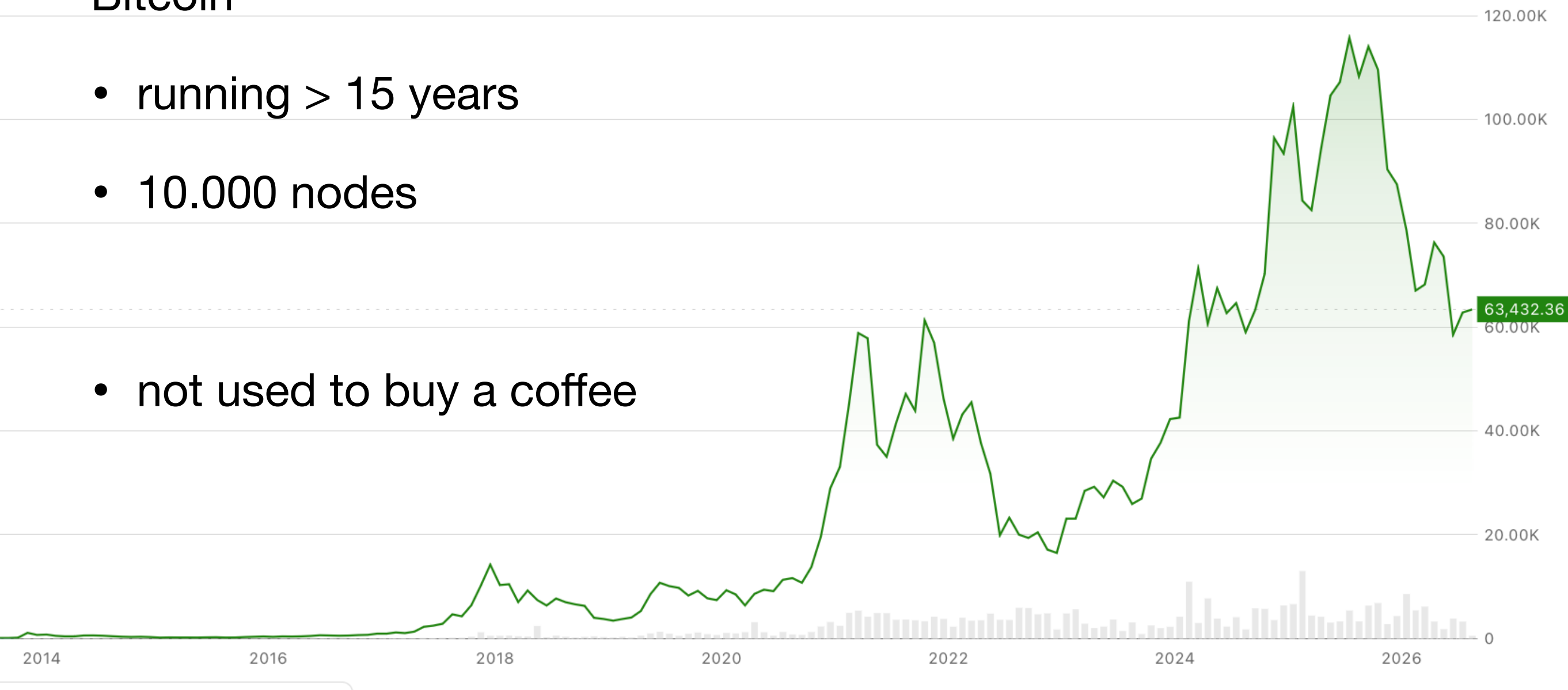
Intro

Blockchain

Motivation

Bitcoin

- running > 15 years
- 10.000 nodes
- not used to buy a coffee



<https://www.coindesk.com/price/bitcoin>

Blockchain

Motivation

Bitcoin

- running > 15 years
- 10.000 nodes
- not used to buy a coffee

How, why does it work?

Is it useful?

Blockchain and applications

Take a look at blockchain technology and applications

- What works
- What does not work

Learning goal: Know how and when to use it.

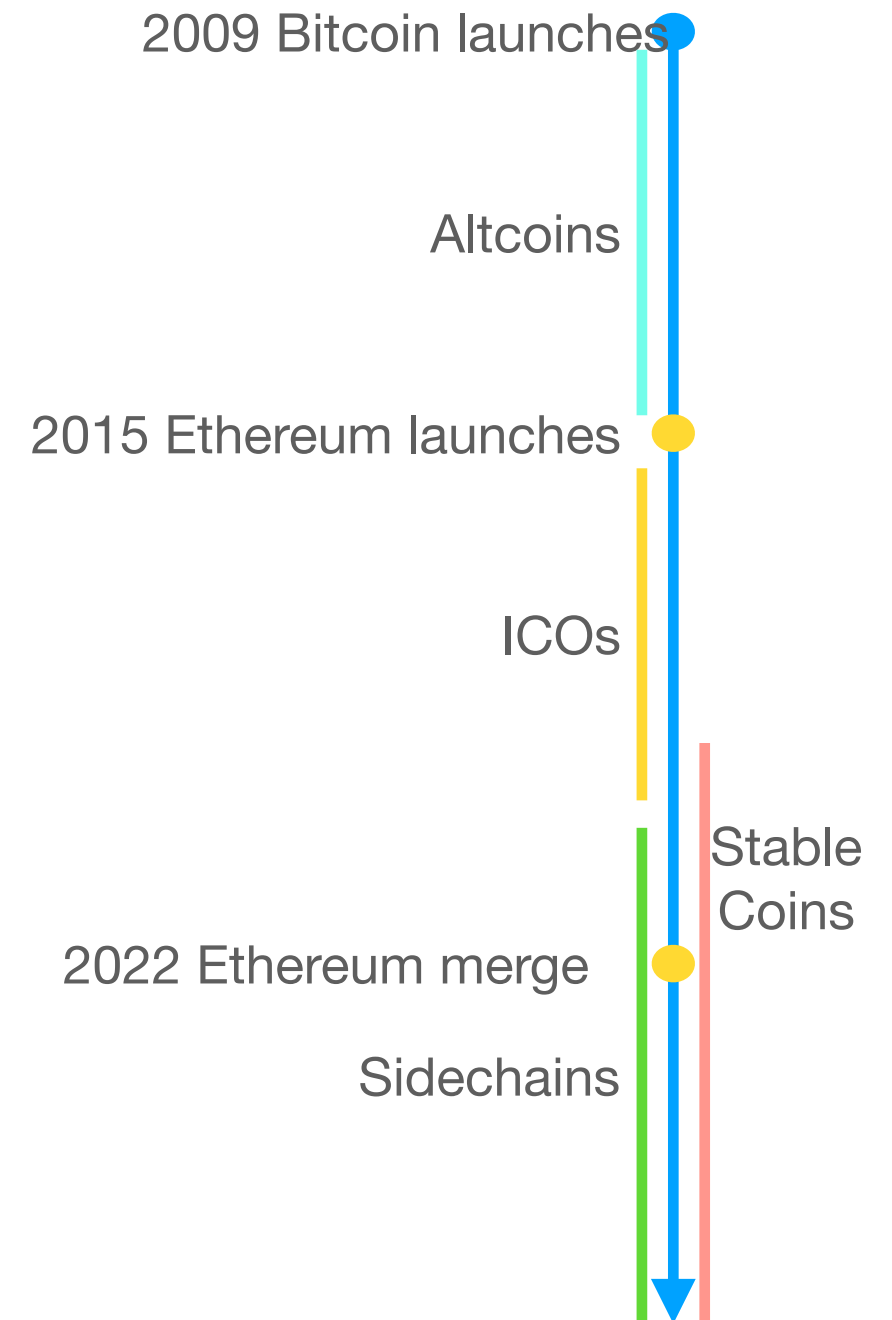
Know when not to use it.

But: No investment tips

Blockchain

Timeline

- 2009 - Bitcoin launch - PoW
- 2015 - Ethereum launches
- 2022 - Ethereum merge - PoS
- 2011 - 2014 Altcoins
- 2017 -> ICO
- 2019 -> Stable coins
- 2022 -> Sidechain, Rollup and Layer 2.



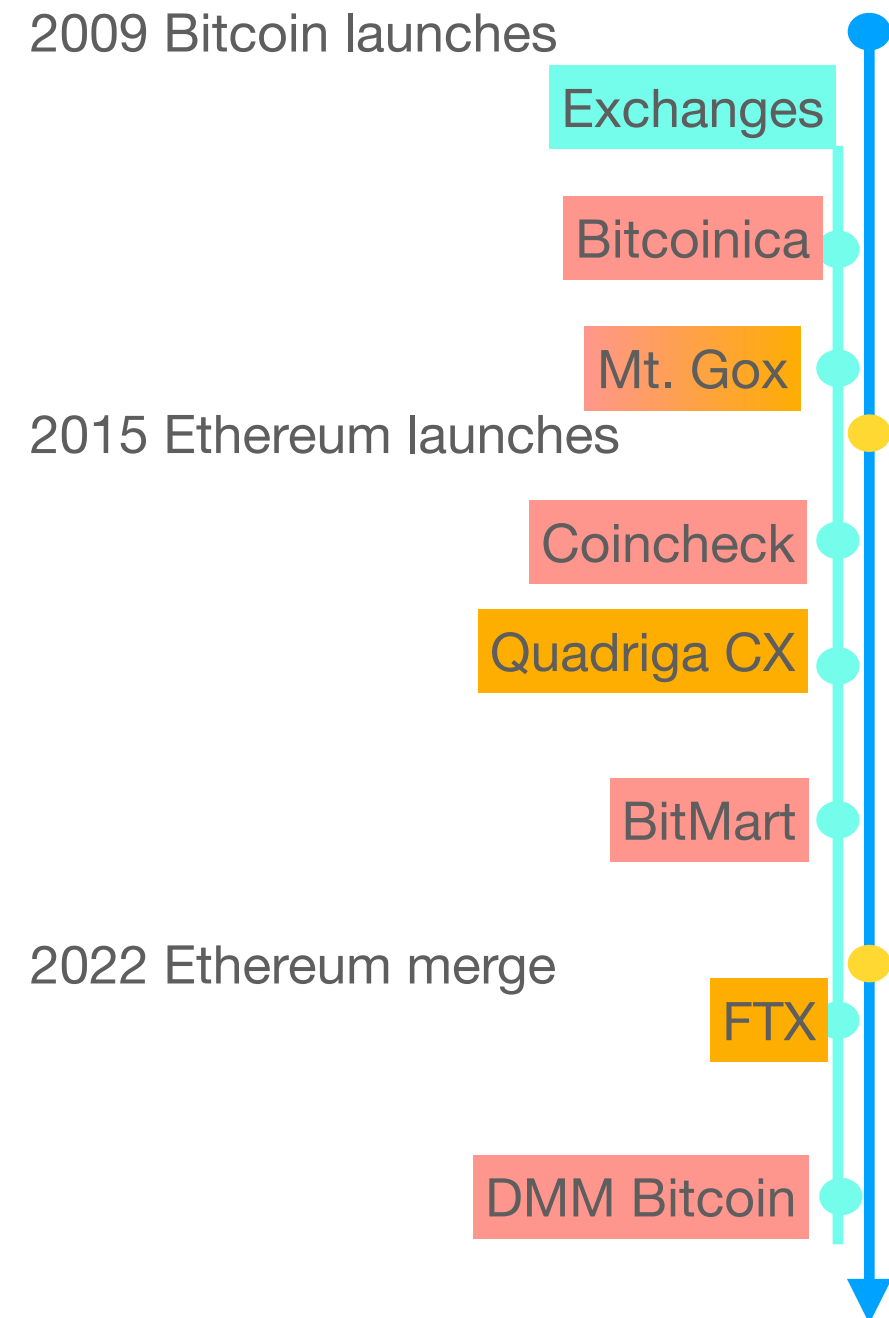
Blockchain

Attack and fraud timeline

- Exchanges used to buy or sell tokens

- Attacks

- Fraud



Blockchain

Attack and fraud timeline

- Fake projects
- too good to be true
- pump and dump meme-coins

2009 Bitcoin launches

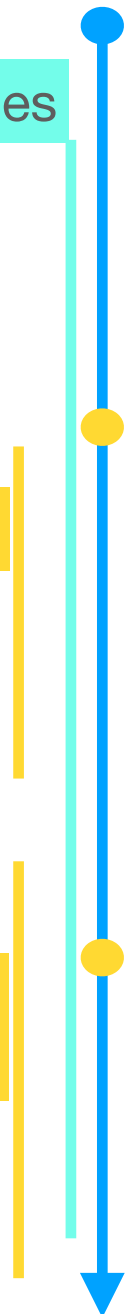
Exchanges

2015 Ethereum launches

ICO scams

2022 Ethereum merge

Meme Coin
manipulation



Blockchain

Attack and fraud timeline

- Smart Contract attacks
- Complex code on top of infrastructure
- Attacks on Oracles

2009 Bitcoin launches

2015 Ethereum launches

2022 Ethereum merge

Cetus

SC Attacks

DAO
Multisig

Exchanges

ICO scam

Meme C
manipula

Blockchain

Attack and fraud timeline

- Attacks on bridges and cross chain
- Complex protocols
- Spanning multiple infrastructures

2009 Bitcoin launches

2015 Ethereum launches

2022 Ethereum merge

Exchanges

ICO scam

SC Attack

Cross chain
attack

Meme C
manipula

Blockchain

Attack and fraud timeline

- Since 2021
 - More then \$12B crypto stolen every year.

2009 Bitcoin launches

Exchanges

2015 Ethereum launches

ICO scam

SC Attack

2022 Ethereum merge

Cross chain
attack

Meme C
manipula

Blockchain

How does it work

- Blockchain datastructures

Blockchain datastructure

What is a blockchain

A blockchain is an append only log
secured against changed.

Typically a blockchain

- is stored on different nodes

Idea: Log all interactions

- Log enables anyone to reproduce/recreate state.

Cryptographic hash function

Idea

$$H(x) = y$$

Cryptographic hash function

Idea

$$H(x) = y$$

- x string or byte array
- y fixed size byte array

looks random: hashing something new gives a random value

is deterministic: hashing something twice gives the same value

Cryptographic hash function

Properties

```
<script src="https://code.jquery.com/jquery-3.3.1.slim.min.js"
integrity="sha384-q8i/X+965DzO0rT7abK41JStQIAqVgRVzpbzo5smXKp4YfRvH+8abtTE1Pi6jizo">
</script>
```

Properties:

- **Pre-image resistance:**
given y cannot find x s.t. $H(x) = y$
- **Weak collision resistance:**
given x cannot find x' s.t. $H(x) = H(x')$
- **Strong collision resistance:**
cannot find x and x' s.t. $H(x) = H(x')$

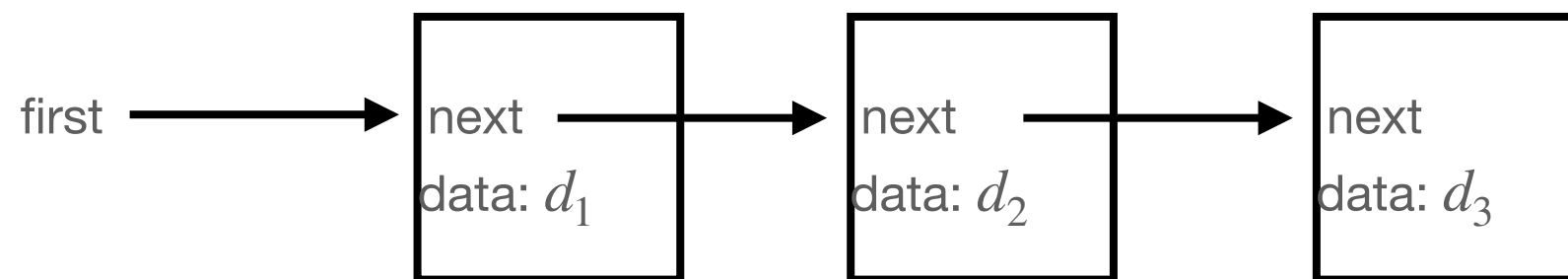
Use cases examples:

- Password hashes
- HTML5 integrity attribute

Hash chain

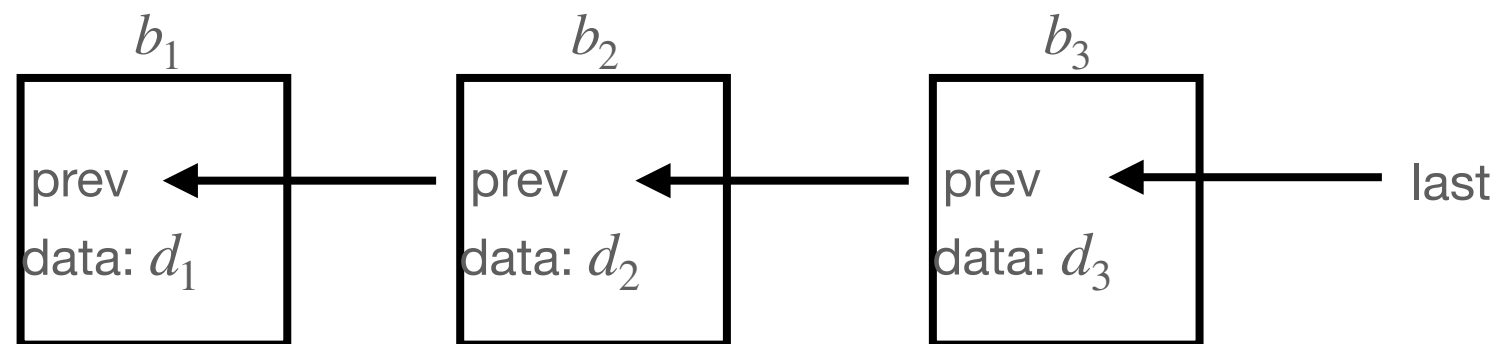
Hash chain

Linked list



```
type Node struct {  
    next pointer  
    data bytes  
}
```

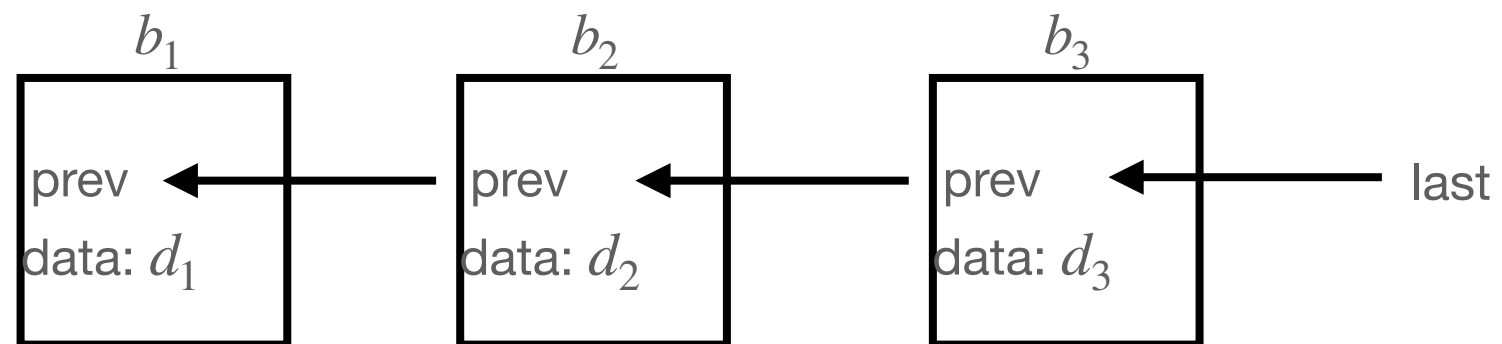
Hash chain



```
type Block struct {  
    prev pointer  
    data bytes  
    prevhash hash  
}
```

- Can hash a block by concatenating fields.
- Blockhash gives id:
 $id_b = H(b.\text{prevhash} || b.\text{data})$
- $b_2.\text{prevhash} = id_{b_1}$

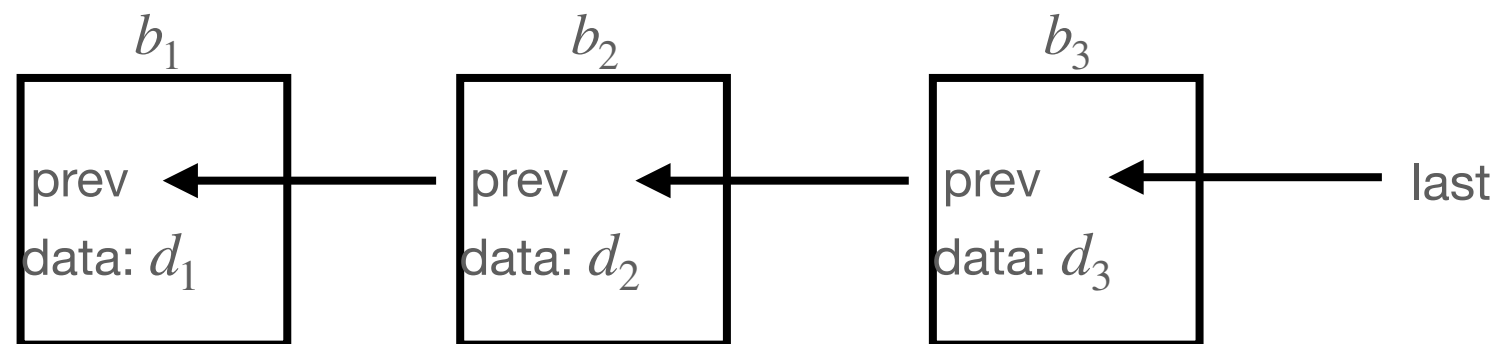
Hash chain



- $id_b = H(b.\text{prevhash} || b.\text{data})$
- Blockchain identified by id_{b_3}
- Changing d_1 changes id_{b_3}
- Removing b_2 changes id_{b_3}
- Adding b'_2 changes id_{b_3}

secured against changes

Hash chain



```
type Block struct {  
    prev pointer  
    data bytes  
    datahash hash  
    prevhash hash  
    timestamp  
}
```

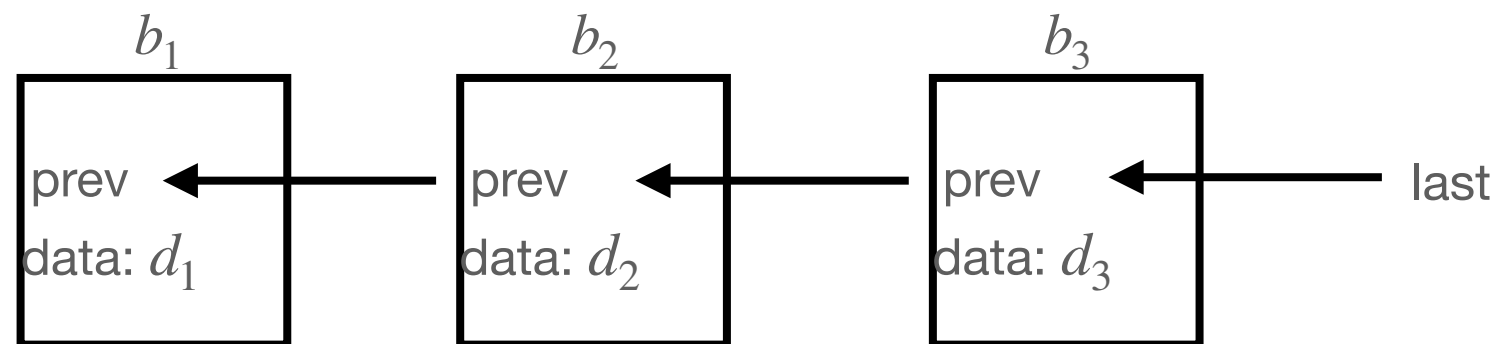
Reduce size:

- $id_b = H(b.\text{prevhash} || b.\text{datahash} || b.\text{ts})$
- $b.\text{datahash} = H(b.\text{data})$

- easy to download d_1 and
- proof that data is included

Hash chain

Example: Linked timestamping



Trusted source collects data and publishes a new block, e.g. on newspaper.



Merkle trees

Merkle trees

Problem

```
type Block struct {  
    prev pointer  
    data bytes  
    datahash hash  
    prevhash hash  
    timestamp  
}
```

Idea:

- put multiple data items into one block

- Is my item in the block?

Merkle trees

Ideas

Data items: $D_1, D_2, D_3, D_4, \dots$

- $\text{datahash} = h$

Data as a hash

- $h = H(D_1 || D_2 || D_3 || \dots)$

Data as a list of hashes

- $h = H(H(D_1) || H(D_2) || H(D_3) || \dots)$

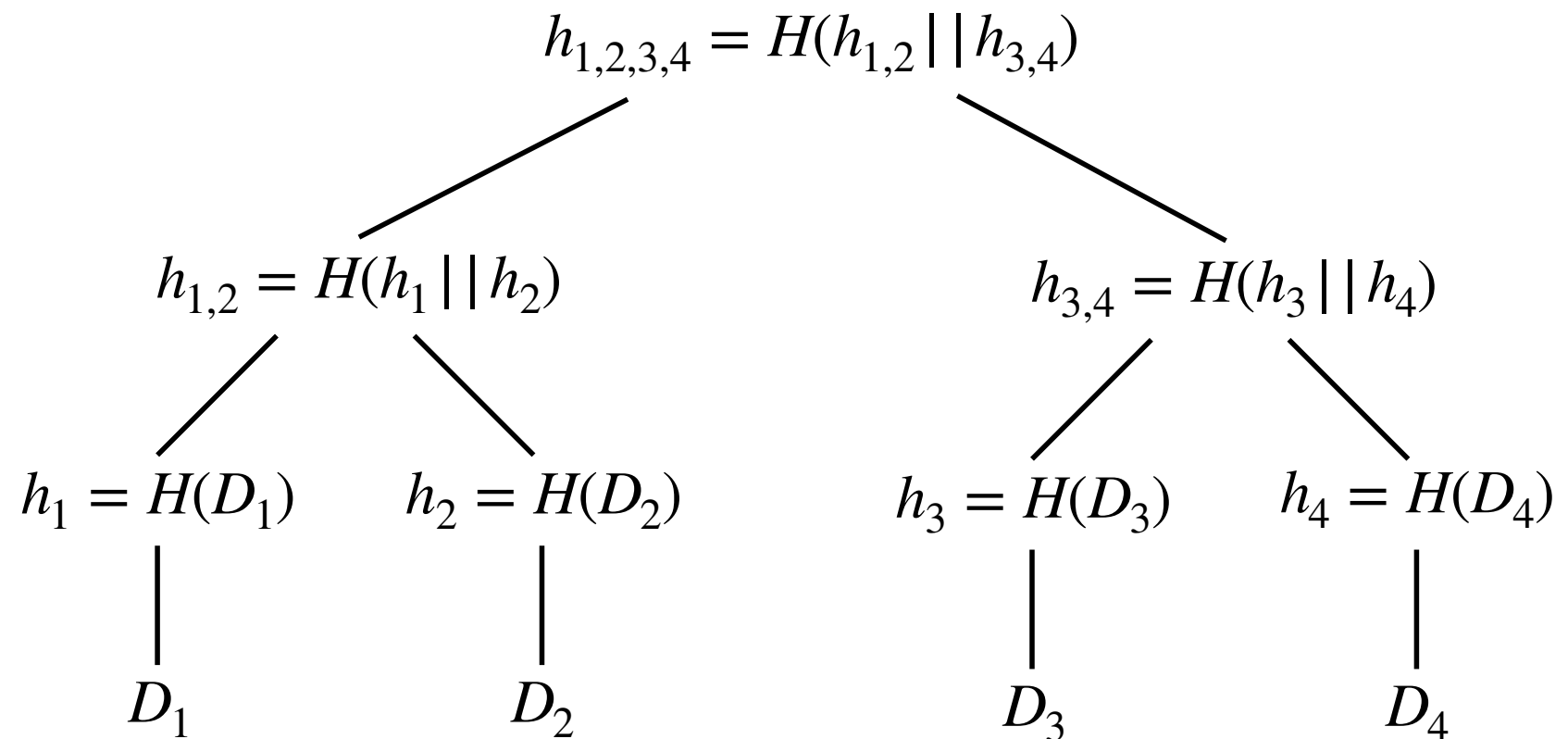
```
type Block struct {  
    prev pointer  
    data bytes  
    datahash hash  
    prevhash hash  
    timestamp  
}
```

Merkle trees

Design

Data items: $D_1, D_2, D_3, D_4, \dots$

- datahash = h

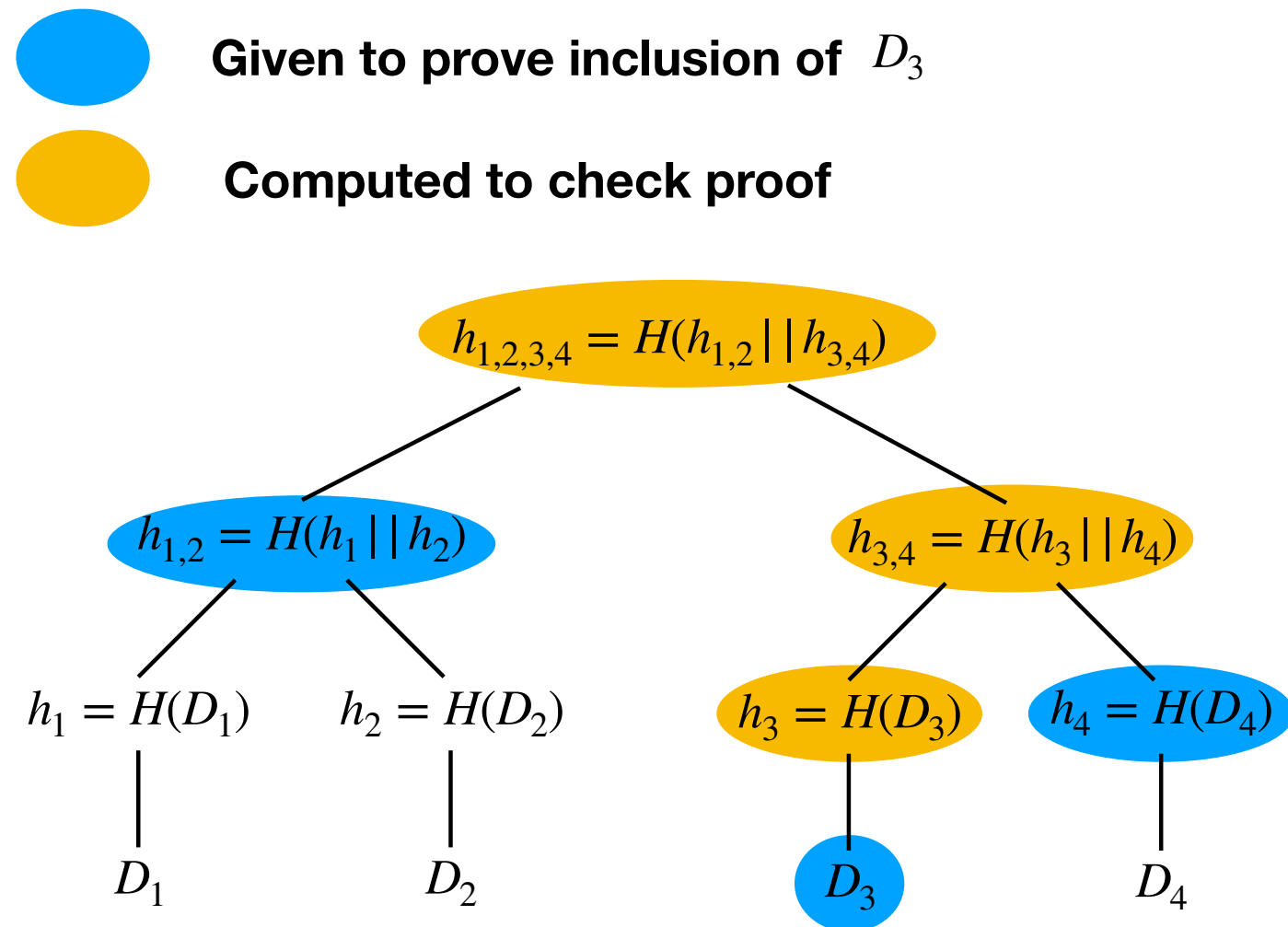


Merkle trees

Proofs

Data items: $D_1, D_2, D_3, D_4, \dots$

- datahash = h



Merkle trees

Proofs

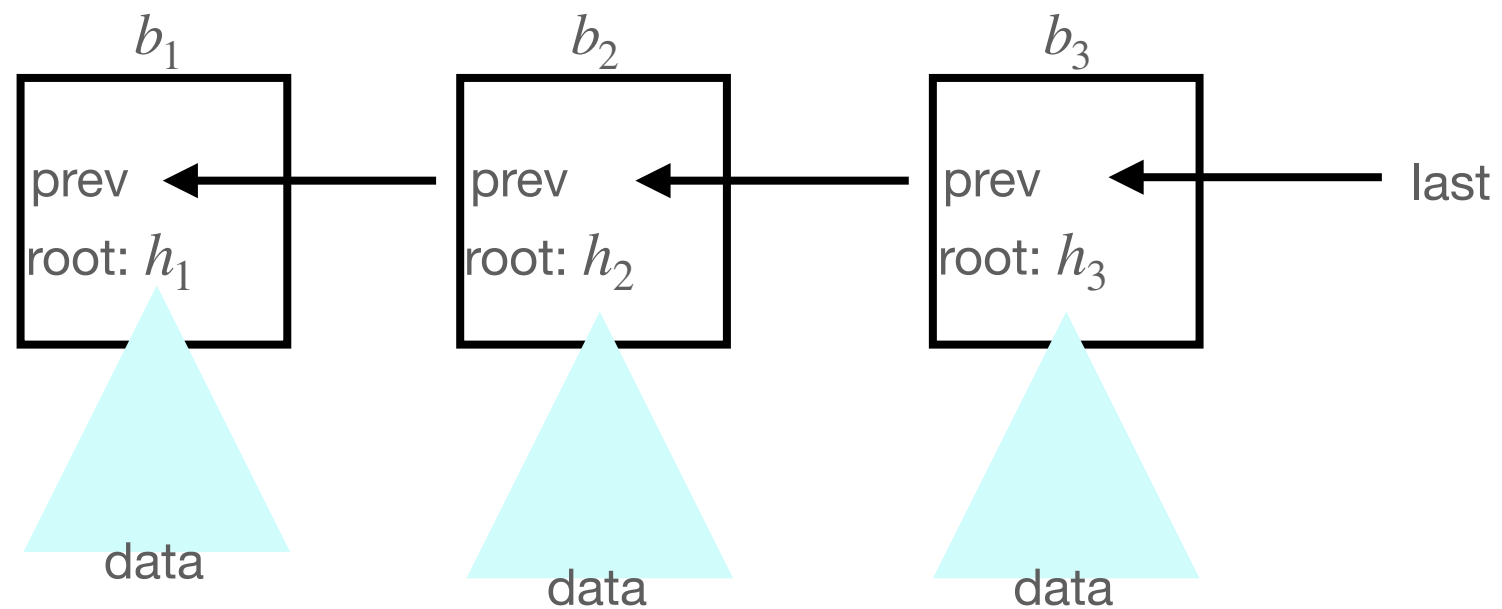
Data items: $D_1, D_2, D_3, D_4, \dots$

- datahash = h

What if we have only 5 data items?

- Duplicate D_5
- Add default element

Hash chain



secured against changes
if we know id_3

How can we update consistently,
if stored on many nodes?